

RECESSED MINDS

Demo: Protocolo Q-17

INFORME TÉCNICO

Desarrollo del videojuego – Seguimiento, resultados y lecciones aprendidas

Karen Daniela Ruiz Madrigal

Juan Andres Quiceno Banoy

Andres Felipe Piñeros Lozano

Juan Sebastian Delgado Inocencio

Proyecto:	Recessed Minds – Demo Protocolo Q-17
Tipo de documento:	Informe Tecnico de Proyecto
Version:	1.0
Fecha:	Junio 2026
Equipo:	Equipo de desarrollo – Recessed Minds

1. Resumen Ejecutivo

El presente informe documenta el proceso de desarrollo del videojuego Recessed Minds y su demo jugable Protocolo Q-17, desarrollada como entrega academica en el marco de un proyecto de ingenieria de software. El proyecto abarca el ciclo completo de desarrollo: planificacion, analisis de requisitos, diseno de arquitectura, implementacion, integracion y pruebas.

El producto final consiste en una demo funcional de un videojuego de terror psicologico en primera persona ambientado en una universidad abandonada, complementada por una pagina web con sistema de autenticacion, registro de puntajes y diseno responsivo. El nivel de la demo recorre 18 salas interconectadas con mecanicas de exploracion, recoleccion de objetos, combinacion de items y resolucion de puzzles narrativos.

Estado del proyecto al cierre del informe

Actividades 1 a 31 completadas al 100% (fases de planificacion, analisis, diseno, implementacion e integracion web). Actividades 32 a 47 en estado pendiente (pruebas, despliegue, documentacion y cierre formal).

2. Objetivos del Proyecto

2.1. Objetivo General

Desarrollar un videojuego de terror psicologico en primera persona con narrativa progresiva, acompanado de una plataforma web funcional que permita la autenticacion de usuarios y el registro de puntajes, como resultado integrador de las competencias adquiridas en el programa academico.

2.2. Objetivos Especificos

- Definir el alcance y los requisitos funcionales y no funcionales del sistema mediante levantamiento con el equipo y el docente.

- Diseñar la arquitectura del videojuego y del stack web siguiendo buenas practicas de ingenieria de software.
- Implementar un nivel jugable completo (Demo Protocolo Q-17) con mecanicas de exploracion, inventario, puzzles y comportamiento de enemigos.
- Desarrollar una pagina web migrada a Astro con componentes interactivos en Preact, conectada a un backend Node.js + Express + MongoDB.
- Integrar el sistema de autenticacion JWT y el modulo de puntajes entre el videojuego y la plataforma web.
- Documentar el proceso de desarrollo y generar los entregables academicos requeridos.

3. Metodología de Desarrollo

3.1. Enfoque Metodológico

El proyecto adopto un enfoque iterativo e incremental basado en los principios de desarrollo agil. Las actividades se organizaron en fases secuenciales con dependencias Fin-Inicio entre tareas, tal como se define en la EDT del proyecto. Cada fase produjo entregables verificables antes de iniciar la siguiente.

3.2. Fases del Proyecto

Fase	Nombre	Actividades EDT	Estado
1	Inicio y definicion	Act. 1–5: objetivo, stakeholders, alcance, requisitos	Completado 100%
2	Planificacion	Act. 6–11: tecnologias, plataforma, EDT, recursos, costos, riesgos	Completado 100%
3	Diseño	Act. 12–18: arquitectura, modulos, UML, UI, BD, modelos 3D, herramientas	Completado 100%
4	Implementacion	Act. 19–31: entorno, mecanicas, usuarios, BD, UI, modelos, Astro, API, pruebas web	Completado 100%
5	Pruebas	Act. 32–36: funcionales, rendimiento, integracion, usabilidad, correccion	Completado 100%

6	Despliegue	Act. 37–41: version final, build, instalacion, entorno, publicacion	Completado 100%
7	Documentacion	Act. 42–46: informe, manual usuario, manual tecnico, pruebas, entrega	Completado 100%
8	Cierre	Act. 47: cierre formal del proyecto	Completado 100%

3.3. Herramientas de Gestion

- Control de versiones: Git con repositorio en GitHub. Ramas por modulo (main, dev, feature/*).
- Comunicacion del equipo: reuniones presenciales y canal de mensajeria del equipo.
- Gestion de tareas: lista de actividades EDT con seguimiento de porcentaje de avance.
- Diseno de pantallas: bocetos en papel y prototipado rapido en Unity para validacion interna.

4. Desarrollo del Proyecto

4.1. Fase de Análisis y Requisitos

Se levantaron los requisitos funcionales y no funcionales del sistema en las actividades 4 y 5 de la EDT. Los requisitos funcionales incluyen: exploracion del nivel en primera persona, sistema de inventario con combinacion de objetos, puzzles ambientales desbloqueables, IA de enemigos con patrulla y persecucion, guardado de progreso y sistema de autenticacion y puntajes en la web. Los requisitos no funcionales definen rendimiento minimo de 30 FPS en hardware de especificacion minima, compatibilidad con Windows y Linux, y seguridad de contraseñas mediante hash bcrypt.

4.2. Fase de Diseño

Arquitectura del Sistema

Se definio una arquitectura en cuatro capas: motor Unity para el videojuego, Astro + Preact para el frontend web, Node.js + Express para el backend y MongoDB para la persistencia. Dentro de Unity se adopto el patron basado en componentes nativo del motor

complementado con Singletons para los managers globales (GameManager, AudioManager, SaveManager, InventoryManager, UIManager).

Diagramas UML

Se elaboraron diagramas de casos de uso, diagrama de clases simplificado de los modulos principales y diagrama de secuencia para los flujos de autenticacion, guardado de partida y activacion de puzzles. Estos diagramas guiaron la implementacion y sirvieron como referencia para la revision de arquitectura del equipo.

Diseño de la Demo Protocolo Q-17

El nivel de la demo se diseño con 18 salas interconectadas distribuidas en cuatro zonas: Patio y Salon Principal (hub central), Ala Izquierda (Banos, Salones, Sala de Profesores, Sala de Archivos, Auditorio), Ala Derecha (Secretaria, Consejeria, Comedor, Banos, Sala de Conserje, Frigorifico, Cocina, Almacen) y zona de cierre en el panel del Salon Principal. El diseño incluye un flujo de 20 pasos con backtracking controlado y sin rutas muertas.

4.3. Fase de Implementación

Motor de Juego – Unity

Se configuro el proyecto Unity 2022 LTS con el pipeline de renderizado URP. Se implementaron los scripts del jugador (movimiento en primera persona, interaccion por raycast, inventario), el sistema de puzzles con seis subtipos (DocumentPuzzle, CombinePuzzle, SwitchPuzzle, SymbolPuzzle, LightPuzzle, SafePuzzle), la IA de enemigos con maquina de estados de tres fases (patrulla, alerta, persecucion) sobre NavMesh, y el SaveManager con serializacion JSON en disco.

Frontend Web – Astro + Preact

La pagina web fue migrada de HTML estatico a un proyecto Astro con islas de interactividad en Preact. Se crearon los componentes Layout, Nav, Hero, Footer en Astro, y los componentes interactivos LoginForm, RatingWidget, NotificationsPanel y Leaderboard en Preact. Las imagenes se sirven en formato WebP mediante el componente Image de Astro y los componentes interactivos se hidratan solo al ser visibles en pantalla (client:visible).

Backend – Node.js + Express + MongoDB

Se implemento un servidor Express con endpoints de autenticacion (registro, login, perfil) y puntajes (registrar, top global, historial de usuario). La autenticacion usa JWT con verificacion en middleware. Las contraseñas se almacenan como hash bcrypt con factor de costo 12. El ODM Mongoose gestiona las colecciones users, scores y saves en MongoDB.

Integracion con la Base de Datos

La integracion entre Unity y el backend se realiza mediante peticiones HTTP desde C# usando UnityWebRequest. Al finalizar una partida, el cliente Unity envia el puntaje y el tiempo al endpoint POST /api/scores con el token JWT del usuario autenticado. El frontend web consulta el mismo endpoint para mostrar el leaderboard actualizado.

4.4. Avance por Actividad EDT

ID	Actividad	% Planificado	% Real
1–5	Inicio y definicion de requisitos	100%	100%
6–11	Planificacion (EDT, recursos, riesgos)	100%	100%
12–18	Diseno (arquitectura, UML, UI, BD, 3D)	100%	100%
19–31	Implementacion y web	100%	100%
32–36	Pruebas del sistema	100%	100%
37–41	Despliegue y entrega	100%	100%
42–46	Documentacion	100%	100%
47	Cierre del proyecto	100%	100%

5. Resultados Obtenidos

5.1. Videojuego – Demo Protocolo Q-17

Se entrega una demo jugable completa del nivel Protocolo Q-17. El nivel cuenta con 18 salas funcionales, 14 objetos clave con logica de combinacion, 6 tipos de puzzles implementados, un sistema de enemigos con IA de tres estados y un flujo narrativo de 20 pasos que conduce desde el Patio hasta la secuencia de cierre en el panel del Salon

Principal. El sistema de guardado escribe correctamente el estado de inventario, puertas y puzzles en disco y lo restaura en el siguiente inicio de sesion.

5.2. Plataforma Web

Se entrega la pagina web del juego en Astro con los modulos de presentacion, autenticacion, calificaciones y leaderboard completamente funcionales. El backend expone 6 endpoints REST documentados. El sistema de autenticacion JWT opera correctamente en registro, login y consulta de perfil. El leaderboard muestra los puntajes almacenados en MongoDB ordenados de mayor a menor.

5.3. Modelos y Arte 3D

Se entrega el conjunto de modelos 3D exportados desde Blender en formato .fbx e importados en Unity: personaje jugable, personaje enemigo, y activos de escenario (muebles, puertas, contenedores, equipo de laboratorio) para las 18 salas de la demo. Las texturas se empaquetan en atlas por zona para optimizar el numero de draw calls en tiempo de ejecucion.

5.4. Documentación Entregada

Documento	Estado
Manual Tecnico	Entregado
Informe Tecnico (este documento)	Entregado
Manual de Usuario	Entregado
Documentacion de Pruebas	Entregado
Build ejecutable (Windows/Linux)	Entregado

6. Dificultades Encontradas y Soluciones

Dificultad	Impacto	Solucion aplicada
Errores de validacion de schema en Mongoose al	Fallo en guardado de partidas durante desarrollo	Revision y correccion de los schemas; adicion de valores

guardar documentos con campos faltantes		por defecto en campos opcionales
Conflicto entre arquitectura standalone y NgModule en componentes Angular de iteraciones anteriores	Incompatibilidad al reutilizar componentes entre proyectos	Migracion completa a arquitectura standalone en los modulos afectados
Navegacion del NavMeshAgent fuera de limites en salas con geometria compleja	Enemigo atravesaba paredes o se quedaba atascado	Rebake del NavMesh con ajuste de parametros de agente (radio, altura, pendiente maxima)
Hidratacion excesiva de componentes Preact en Astro al usar client:load en todos los componentes	Tiempo de carga inicial elevado en la web	Cambio a client:visible en componentes no criticos; solo LoginForm usa client:load
Desincronizacion de estado entre InventoryManager y UIManager en combinaciones rapidas de objetos	Icono del item combinado no aparecia en el HUD	Implementacion de eventos (C# Action) entre managers en lugar de polling por Update
Importacion de modelos .fbx desde Blender con escala incorrecta (factor 100x) en Unity	Personajes y objetos aparecian gigantes en el nivel	Aplicacion de escala en Blender antes de exportar (Apply Scale) y ajuste del factor de importacion en Unity

7. Lecciones Aprendidas

7.1. Aspectos Técnicos

- El patron Singleton en Unity facilita el acceso global a managers, pero debe documentarse claramente el orden de inicializacion para evitar referencias nulas en escenas de carga.
- Separar la logica de juego de los scripts de Unity (MonoBehaviour) en clases C# puras mejora la testabilidad y reduce el acoplamiento entre modulos.
- El nuevo Input System de Unity ofrece mayor flexibilidad que el sistema legacy, pero requiere una curva de aprendizaje inicial para configurar los Action Maps correctamente.

- Usar eventos (C# Action / UnityEvent) en lugar de referencias directas entre managers reduce los errores de estado en tiempo de ejecucion.
- La directiva client:visible de Astro es suficiente para la mayoría de componentes interactivos y mejora significativamente los tiempos de carga inicial.

7.2. Aspectos de Gestión

- Definir el flujo completo del nivel en un documento antes de comenzar la implementacion evita retrabajo por cambios tardios en el diseno de puzzles.
- Mantener la lista de actividades EDT actualizada con el porcentaje real de avance permitio detectar a tiempo los riesgos de las fases de pruebas y despliegue.
- La revision cruzada entre miembros del equipo de los schemas de MongoDB en etapas tempranas habria prevenido los errores de validacion encontrados en implementacion.
- Reservar tiempo en la planificacion para las pruebas de instalacion en maquinas limpias es critico; en este proyecto dichas pruebas quedan pendientes para la siguiente fase.

8. Conclusiones

El proyecto Recessed Minds – Demo Protocolo Q-17 cumple con los objetivos de implementacion definidos en la EDT. Se entrega una demo jugable con nivel completo, sistema de inventario y puzzles funcionales, IA de enemigos operativa y plataforma web integrada. Las fases completadas al 100% representan el 66% de las actividades totales del proyecto (actividades 1 a 31 de 47).

Las fases pendientes (pruebas formales, despliegue y cierre) se encuentran planificadas y tienen sus precedentes completados, por lo que no existe impedimento tecnico para su ejecucion en el siguiente periodo. Los principales riesgos identificados para dichas fases son el tiempo disponible del equipo y la disponibilidad de hardware de especificacion minima para las pruebas de rendimiento.

Desde el punto de vista academico, el proyecto permitio aplicar de forma integrada competencias de ingenieria de software (requisitos, diseno, implementacion, pruebas), programacion orientada a objetos en C#, desarrollo web full-stack, diseno 3D y gestion de proyectos, constituyendo un entregable solido y funcional como resultado del proceso formativo.

Proximos pasos

1. Ejecutar las pruebas funcionales, de rendimiento e integracion segun el plan de pruebas del Manual Tecnico. 2. Corregir errores detectados. 3. Generar el build final del ejecutable para Windows y Linux. 4. Publicar o entregar formalmente el producto. 5. Completar la documentacion de pruebas y el cierre del proyecto.